# Simple Digital Image Processing Techniques and Suitable Features for Three Practical Cases

Student Name
Digital Image Processing Course
University / Department

***Abstract*—Digital image processing can solve practical problems without requiring complex machine-learning models. This report examines three introductory cases: counting coins, enhancing a scanned document, and removing impulse noise from a photograph. For each case, a simple processing pipeline is implemented and the image features supporting that pipeline are identified. The accompanying Python notebook uses publicly accessible sample images and produces all reported figures and measurements. Global thresholding detects 21 objects against a manual reference of 24 coins, illustrating the limitations of a basic segmentation baseline. Local thresholding reduces the document's foreground fraction from 35.68% to 14.77%, although this is not an accuracy measurement. For the simulated noise experiment, a median filter improves peak signal-to-noise ratio from 17.77 to 30.12 dB. These examples show that useful features depend on the task, image conditions, and evaluation method.**

***Index Terms*—digital image processing, feature selection, thresholding, connected components, median filtering**

## I. Introduction

Digital image processing transforms an image into a more useful representation or extracts information from its pixels. Typical applications include automatic counting, document preparation, and image restoration. A suitable technique should be chosen from the properties of the problem rather than from its complexity. For example, a coin's area may be useful for counting objects, but it does not directly explain how to remove noise from a photograph.

This report considers three cases that are practical for an introductory course. The goals are to summarize each problem, identify appropriate features, implement understandable methods, and discuss actual results and limitations. The implementation uses NumPy, Matplotlib, and scikit-image, a documented Python image-processing library [1]. No training data, neural network, or graphics processing unit is required.

## II. Data and Experimental Procedure

Three images from the official scikit-image collection are used [2]: `coins` ($303 \times 384$ pixels), `page` ($191 \times 384$), and `camera` ($512 \times 512$). The collection was located through a web search and the samples were obtained through the original library, rather than downloading untraceable search-engine thumbnails. Coins are credited to the Brooklyn Museum collection with no known copyright restrictions; the current camera image is CC0 by Lav Varshney. The scanned page is supplied by scikit-image without a separately identified photographer in its API entry; no separate public-domain claim is made for it. The repository retains source links and the upstream license notice.

All images are converted to floating-point grayscale values in $[0, 1]$. The notebook records package versions, saves the original images, exports comparison figures, and writes measurements to a JSON file. Parameters are fixed for these examples, not optimized on a held-out dataset. The noise experiment uses NumPy's random generator with seed 42. The results are therefore small demonstrations, not evidence of performance across a population of images.

## III. Case I: Counting Coins

### A. Problem and Suitable Features

Counting visible objects is useful in inventory inspection and simple automation. Here, the task is to count separated coins on a relatively dark background, not to recognize their monetary value. Grayscale intensity provides an initial foreground/background cue. Connected-component area helps remove tiny regions, while the centroid gives each detected object's location. Circularity describes how closely a component resembles a circle:

$$C = \frac{4\pi A}{P^2}, \tag{1}$$

where $A$ is area and $P$ is perimeter. An ideal continuous circle has $C = 1$; discrete boundaries and segmentation errors affect the measured value. Circularity is exported as a descriptive feature, not used as an additional rejection threshold in this baseline.

### B. Processing Method

The image is smoothed with a Gaussian filter of standard deviation 1 pixel. Otsu's method selects a global threshold by separating gray-level classes [3]. Pixels above the threshold are treated as foreground. Regions smaller than 120 pixels are removed and holes below 120 pixels are filled. Components touching the image border are discarded, after which connected components are labeled. Each remaining label is counted as one candidate coin. These area settings are specific to the current image resolution.

### C. Results and Limitations

The measured threshold is 0.4073 and the method returns 21 objects. A manual count of the original image gives 24 coins, so the absolute count error is 3 and the relative count error is $3/24 = 12.5\%$. This is a count discrepancy, not a

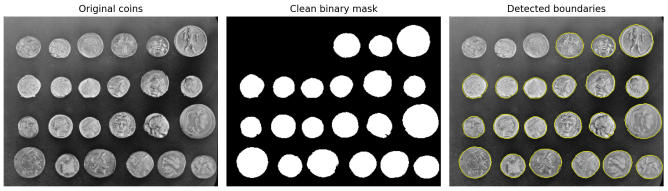



Fig. 1. Coin-counting baseline: original image, cleaned foreground mask, and detected boundaries. Bright background areas complicate global thresholding.

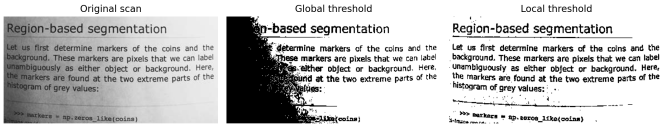



Fig. 2. Document enhancement: original scan, global thresholding, and local thresholding. A spatially varying threshold better follows the background brightness in this example.

precision, recall, or object-detection accuracy score. The mask in Fig. 1 shows why a global threshold is imperfect: some upper coins are connected to bright background regions that are subsequently removed by border clearing.

The baseline is useful because its failure can be explained. Uneven illumination violates the assumption of separable foreground intensities. Touching coins could also produce one component instead of two, and genuine edge objects would be removed by border clearing. An improved exercise could try local segmentation or watershed separation, but these are not necessary to understand the current introductory pipeline. A reliable system would need annotated object masks or centers across many images.

## IV. Case II: Enhancing a Scanned Document

### A. Problem and Suitable Features

Scanned documents often have shadows or uneven illumination that make fixed thresholding unreliable. The goal is to separate dark text from paper before a possible optical character recognition (OCR) step. Relevant features are local intensity, local contrast, connected-component size, and the fraction of pixels marked as text. Local features are appropriate because the brightness of the paper varies spatially.

### B. Processing Method

Two binarization methods are compared: a single global Otsu threshold and a spatially varying Gaussian-weighted threshold [4]. The local method uses a $35 \times 35$ neighborhood and an offset of 0.03:

$$T(x, y) = \mu_w(x, y) - 0.03, \tag{2}$$

where $\mu_w$ is the weighted local mean. A pixel is marked as text when $I(x, y) < T(x, y)$. The binary results are displayed with black text on a white background. The text fraction is calculated directly from the foreground mask, before display inversion.

### C. Results and Limitations

The foreground fraction changes from 35.68% with the global method to 14.77% with the local method. The local mask contains 291 connected components, with a median area of 35 pixels. Visual inspection of Fig. 2 supports the interpretation that local thresholding rejects more of the shaded background while retaining much of the printed text.

However, a lower foreground fraction does not prove better recognition: lost letters also reduce that fraction. Connected components are not equivalent to characters because dots, punctuation, and joined strokes form separate or merged regions. No ground-truth binary mask or transcription is supplied, so pixel accuracy and OCR accuracy are not reported. Important failure cases include thin strokes, textured paper, and text larger than the chosen neighborhood. Practical evaluation should measure character or word error rates on transcribed pages, in addition to inspecting the image.

## V. Case III: Removing Impulse Noise

### A. Problem and Suitable Features

Impulse corruption introduces isolated black and white pixels, sometimes called salt-and-pepper noise. Extreme intensity and disagreement with neighboring pixels are useful cues. The neighborhood median is an appropriate local statistic because it is less affected by isolated extremes than an arithmetic average. This task therefore uses local intensity consistency rather than object shape.

### B. Processing and Evaluation

For each clean-image pixel, a uniform random value is generated. Values below 0.025 set the pixel to black and values above 0.975 set it to white, giving an expected corruption proportion of 5%. The original image remains available as a reference. A Gaussian filter with standard deviation 1 pixel and a median filter with a $3 \times 3$ footprint are applied to the same noisy image.

For $N$ pixels, mean squared error and peak signal-to-noise ratio are

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} (I_i - \hat{I}_i)^2, \tag{3}$$

$$\mathrm{PSNR} = 10 \log_{10} \left( \frac{1}{\mathrm{MSE}} \right), \tag{4}$$

where $I$ is the clean reference and $\hat{I}$ is the tested image. The peak value is 1 because the intensity range is normalized. These reference-based metrics evaluate restoration; they are not features available to a real filter when the clean image is unknown.

### C. Results and Limitations

Table I shows that median filtering provides the lowest MSE and highest PSNR in this controlled experiment. Its PSNR improvement over the noisy image is 12.35 dB, compared with 7.91 dB for Gaussian filtering. This agrees with the noise model: averaging spreads extreme values into nearby pixels,

TABLE I
MEASURED RESTORATION RESULTS FOR ONE IMAGE AND SEED

| Image | MSE | PSNR (dB) |
|---|---|---|
| Noisy input | 0.016707 | 17.77 |
| Gaussian filtered | 0.002704 | 25.68 |
| Median filtered | 0.000973 | 30.12 |

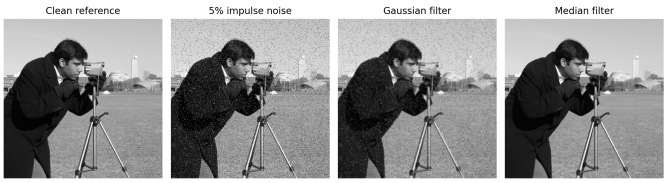



Fig. 3. Clean reference, simulated impulse noise, Gaussian-filter output, and median-filter output. Median filtering rejects isolated extremes but can smooth fine details.

while the median can reject isolated outliers. Fig. 3 provides a qualitative comparison.

Both filters can remove small details. Dense impulses can overwhelm the median, and Gaussian filtering may be more appropriate for a different noise distribution. The numerical ranking applies only to the tested image, corruption model, parameters, and seed. In real photographs, a clean reference is usually unavailable; no-reference inspection or a separate controlled test set would then be needed.

## VI. DISCUSSION: MATCHING FEATURES TO THE TASK

The three cases illustrate distinct feature roles. In coin counting, intensity creates candidate regions and geometric features describe them. In document enhancement, local brightness and contrast support pixel classification, while component size and text fraction provide diagnostics. In denoising, neighborhood statistics guide restoration and reference errors quantify its outcome. A diagnostic statistic should not be confused with an accuracy score or an inference-time feature.

The simplest method is not always the most accurate, but it is useful when its assumptions and errors are visible. Image scale, illumination, noise type, and object overlap all affect parameter choices. A fair extension would include more samples, annotated references where appropriate, and parameter selection on different data from the final evaluation set. The notebook deliberately keeps these distinctions explicit instead of presenting a successful-looking output as proof of general reliability.

## VII. CONCLUSION

Three accessible image-processing pipelines were implemented using basic operations. Intensity and geometry support coin counting, local contrast supports document binarization, and neighborhood statistics support impulse-noise removal. The experiments demonstrate both usefulness and limitations: coin counting underestimates the reference, document statistics require cautious interpretation, and median filtering performs well for the simulated impulse model. Selecting features that match the physical problem is more important than choosing an unnecessarily advanced algorithm. The accompanying executable notebook and saved measurements make these introductory observations reproducible.

## REFERENCES

[1] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.

[2] scikit-image contributors, "Sample images: skimage.data, version 0.24," https://scikit-image.org/docs/0.24.x/api/skimage.data.html, accessed: Nov. 9, 2026.

[3] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[4] scikit-image contributors, "Thresholding: skimage.filters, version 0.24," https://scikit-image.org/docs/0.24.x/api/skimage.filters.html, accessed: Nov. 9, 2026.